ASSIGNMENT 5 – CO5

PYTHON PROGRAMMING

DATA ACQUISITION, CLEANING, ANALYSIS AND AI APPLICATIONS

Name: S.THABASHYA SRI

Register Number: 192520015

Subject: Python Programming

Course Outcome: CO5

GitHub Repository Link: https://github.com/thabashyasri/CSA0823-PYTHON-PROGRAMMING

Department:B.Tech BIOINFORMATICS

ASSIGNMENT 5 – CO5

Python Programming – Data Acquisition, Cleaning, Analysis and AI Applications

PART A – DATA AND PROGRAM DESIGN

TASK 1 – DESIGN THE DATA ACQUISITION (A1)

A1.1 – Inspect the Source

A station CSV file contains air-quality readings collected from different monitoring stations.

Column	Description	Example
Station	Station name	Adyar
Date	Reading date	2026-10-01
Hour	Reading hour	10
PM2.5	Fine particulate matter	48
PM10	Coarse particulate matter	80
NO	Nitric oxide reading	15
Temperature	Temperature value	30

Expected data-quality problems:

- Missing values – blank PM2.5 readings.
- NA values – readings recorded as NA.
- Negative values – PM2.5 = -5.
- Extremely large values – PM2.5 = 1500.
- Duplicate records – same station, date and hour repeated.
- Incorrect temperature units – some stations report Fahrenheit instead of Celsius.
- Incorrect data types – numbers stored as text.
- Incorrect date formats – 01/10/2026 instead of 2026-10-01.

A1.2 – Design the Storage

SQLite is used to store station details and air-quality readings.

Table	Column	Data type	Description
Stations	station_id	INTEGER	Primary key
Stations	station_name	TEXT	Station name
Stations	location	TEXT	Station location
Stations	temperature_unit	TEXT	Celsius or Fahrenheit
Readings	reading_id	INTEGER	Primary key
Readings	station_id	INTEGER	Foreign key
Readings	date	TEXT	Reading date
Readings	hour	INTEGER	Reading hour
Readings	pm25	REAL	PM2.5 value
Readings	pm10	REAL	PM10 value
Readings	no_value	REAL	NO reading
Readings	temperature	REAL	Temperature in Celsius

- The Stations table stores station information.
- The Readings table stores daily air-quality readings.
- station_id connects both tables.
- Primary keys uniquely identify records.
- Foreign keys maintain relationships between tables.

A1.3 – CSV or SQL?

Board's question	CSV method	SQL method
Average PM2.5 of Adyar in October	Read and filter all CSV records	Use SELECT, AVG and WHERE
Station with highest PM2.5	Read all records and compare values	Use MAX and GROUP BY
Daily average for each station	Calculate using Python loops	Use AVG and GROUP BY

Conclusion: CSV is useful for storing raw data and simple file exchange. SQL is better for large datasets because it supports filtering, grouping, indexing and faster repeated queries.

Alternative Concept and Complexity Analysis

- Python reads CSV files using pandas.read_csv().
- Data is loaded into a DataFrame.
- Filtering and calculations are performed using pandas.
- Every new question may require scanning the data again.

Operation	Complexity
Full CSV scan	O(n)
Indexed SQL query	Approximately O(log n) for index lookup
CSV storage	O(n) disk space
DataFrame storage	O(n) memory space

SQL is preferable for repeated queries and large datasets because it supports indexing and structured storage.

Runnable Python Demonstration – TASK 1 (A1)

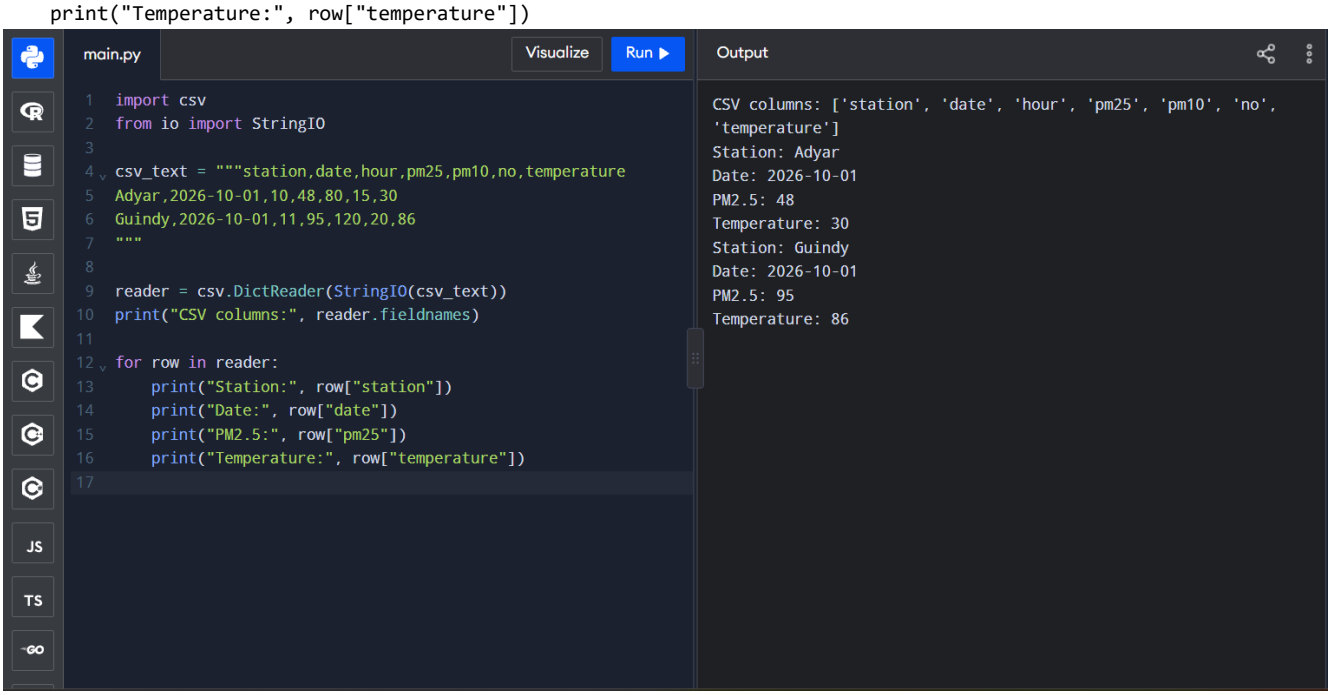
```
# TASK A1 – Python demonstration: read a CSV and inspect its structure
import csv
from io import StringIO

csv_text = """station,date,hour,pm25,pm10,no,temperature
Adyar,2026-10-01,10,48,80,15,30
Guindy,2026-10-01,11,95,120,20,86
"""

reader = csv.DictReader(StringIO(csv_text))
print("CSV columns:", reader.fieldnames)

for row in reader:
    print("Station:", row["station"])
    print("Date:", row["date"])
    print("PM2.5:", row["pm25"])
```

```
print("Temperature:", row["temperature"])
```



TASK 2 – DESIGN THE CLEANING AND NORMALISATION (A2)

A2.1 – Cleaning Rules

- Missing values: Fill missing readings using the previous valid reading (forward fill). If no previous reading exists, use the station median.
- Impossible values: Remove negative PM2.5 values and values above the accepted maximum of 999.
- Duplicate rows: Remove duplicate records using station, date and hour.
- Fahrenheit temperature: Convert Fahrenheit into Celsius using C = ((F - 32) × 5) / 9.

Justification: These rules improve data accuracy and prevent incorrect air-quality analysis.

A2.2 – Normalisation

1. Min-Max Normalisation: X' = (X - Xmin) / (Xmax - Xmin). It converts values into a range between 0 and 1. Example: 20, 40 and 60 become 0, 0.5 and 1.

2. Z-Score Standardisation: Z = (X - μ) / σ. Here, X is the original value, μ is the mean and σ is the standard deviation.

- Min-max is useful when values need a fixed range.
- Z-score is useful when comparing values with different distributions.

A2.3 – Formatting

- Date: YYYY-MM-DD.
- Station name: Title Case (Adyar, Guindy).
- Numbers: Floating-point format with two decimal places.
- Temperature: Celsius.
- Missing values: Represented consistently as NULL in the database.

Alternative Concept and Complexity Analysis

If missing readings are dropped, rows containing missing values are removed and daily averages are calculated using the remaining readings. This may become less representative if many readings are missing.

Method	Time complexity
Dropping missing values	O(n)
Forward filling	O(n)
Median using sorting	O(n log n)

Dropping missing values uses less memory for the cleaned dataset, but it reduces the number of available observations. Forward filling preserves more records but can repeat old measurements.

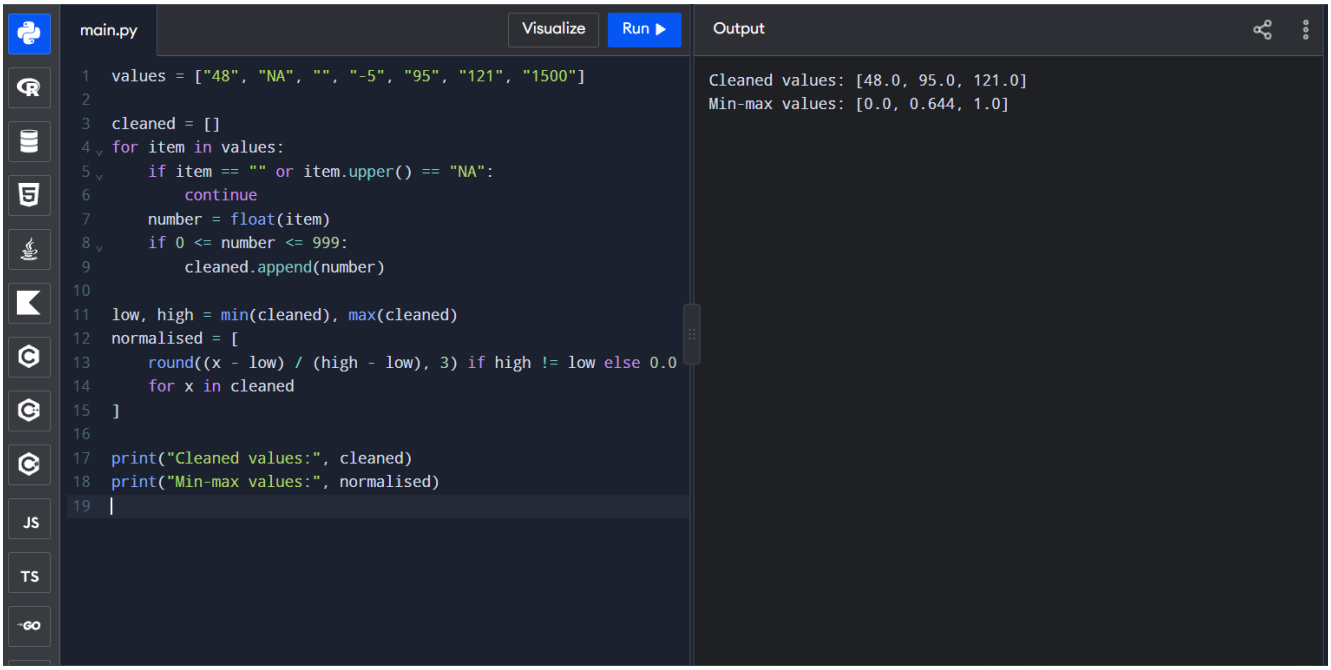
Runnable Python Demonstration – TASK 2 (A2)

```
# TASK A2 – Python demonstration: clean values and normalise PM2.5
values = ["48", "NA", "", "-5", "95", "121", "1500"]

cleaned = []
for item in values:
    if item == "" or item.upper() == "NA":
        continue
    number = float(item)
    if 0 <= number <= 999:
        cleaned.append(number)

low, high = min(cleaned), max(cleaned)
normalised = [
    round((x - low) / (high - low), 3) if high != low else 0.0
    for x in cleaned
]

print("Cleaned values:", cleaned)
print("Min-max values:", normalised)
```



TASK 3 – DESIGN THE ANALYSIS AND VISUALISATION (A3)

A3.1 – Choose the Charts

Board's question	Chart type	Reason
PM2.5 trend over a month	Line chart	Shows changes over time
Comparison of stations	Bar chart	Compares different stations
Temperature vs PM2.5	Scatter plot	Shows relationships between two variables

Hours with worst air	Histogram or bar chart	Shows distribution or hourly differences

- Line charts identify pollution trends.
- Bar charts help identify stations with high pollution.
- Scatter plots show whether temperature and PM2.5 are related.
- Histograms help identify frequently occurring pollution levels.

A3.2 – Design the Summary Table

Column	Description
Station	Station name
Date	Reading date
Minimum	Minimum PM2.5
Maximum	Maximum PM2.5
Mean	Average PM2.5
Valid readings	Number of valid readings

- Read cleaned readings from SQLite.
- Group the readings by station and date.
- Calculate minimum, maximum and mean.
- Count valid readings.
- Store the summary in a separate table.

Alternative Concept and Complexity Analysis

If Python loops are used instead of SQL GROUP BY, read all records, create a dictionary for each station and date, calculate minimum, maximum, total and count, and then calculate the mean.

Method	Time complexity	Space complexity
SQL GROUP BY	O(n) approximately	Database-dependent
Python loops	O(n)	O(g), where g is the number of groups

SQL GROUP BY is more suitable for database-based reporting. Python loops are useful for custom calculations.

Runnable Python Demonstration – TASK 3 (A3)

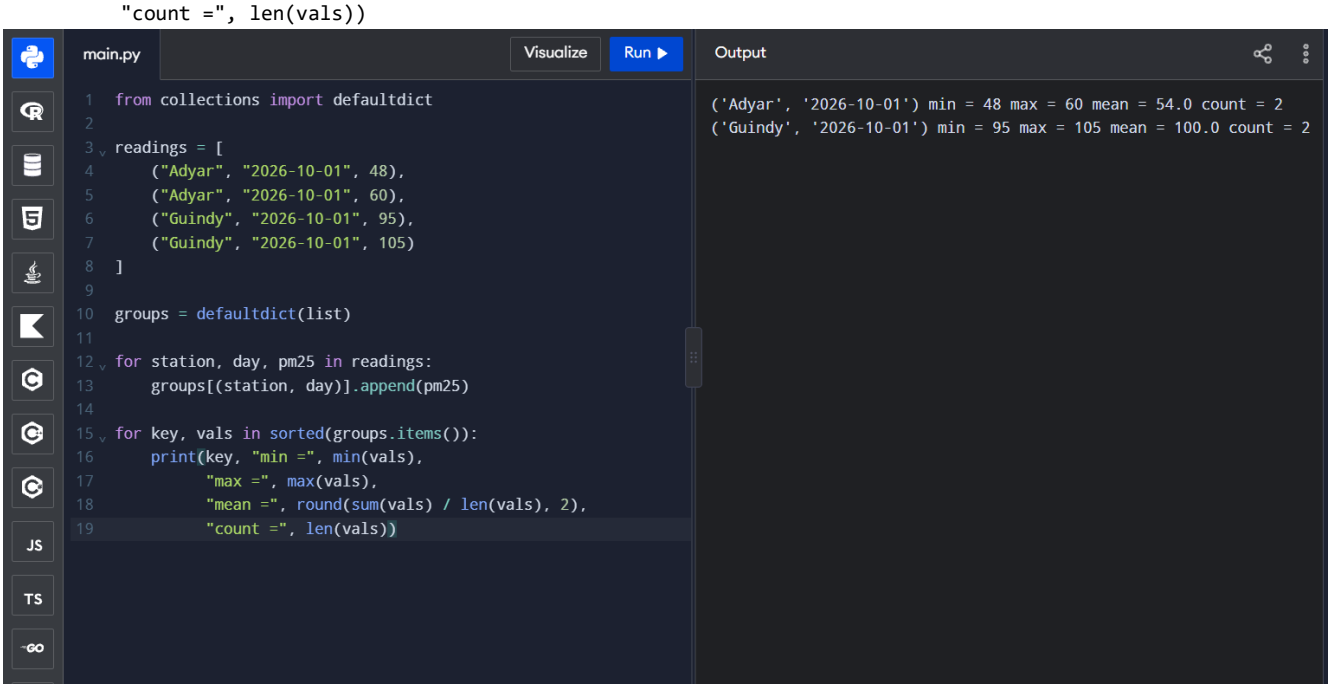
```
# TASK A3 – Python demonstration: daily summary using grouped readings
from collections import defaultdict

readings = [
    ("Adyar", "2026-10-01", 48),
    ("Adyar", "2026-10-01", 60),
    ("Guindy", "2026-10-01", 95),
    ("Guindy", "2026-10-01", 105)
]

groups = defaultdict(list)
for station, day, pm25 in readings:
    groups[(station, day)].append(pm25)

for key, vals in sorted(groups.items()):
    print(key, "min =", min(vals),
          "max =", max(vals),
          "mean =", round(sum(vals) / len(vals), 2),
```

```
        "count =", len(vals))
```



TASK 4 – PLAN DEBUGGING AND PROFILING (A4)

A4.1 – Debugging Plan

- Print statements: Display input data, check intermediate cleaning results and identify unexpected values.
- Assert statements: Verify important conditions. Example: assert pm25 >= 0.
- Logging: Record errors and warnings, maintain execution information and help identify problems.
- PDB debugger: Use breakpoints, execute code line by line, inspect variables and identify errors.

A4.2 – Profiling Plan

- time.perf_counter(): Measures program execution time.
- timeit: Measures how long a small piece of code takes to execute.
- cProfile: Identifies functions that consume the most execution time.

Profiling procedure: Execute the cleaning program, record its execution time, profile it using cProfile, identify the slowest function, optimise it and compare the results.

Alternative Concept and Complexity Analysis

Using print statements for all debugging can slow execution and make messages difficult to organise. Logging is better because DEBUG, INFO, WARNING and ERROR levels can be selected. Both logging and print statements add some execution overhead. A debugger is useful for inspecting specific errors.

Runnable Python Demonstration – TASK 4 (A4)

```
# TASK A4 – Python demonstration: assertions, logging and timing
import logging
import time

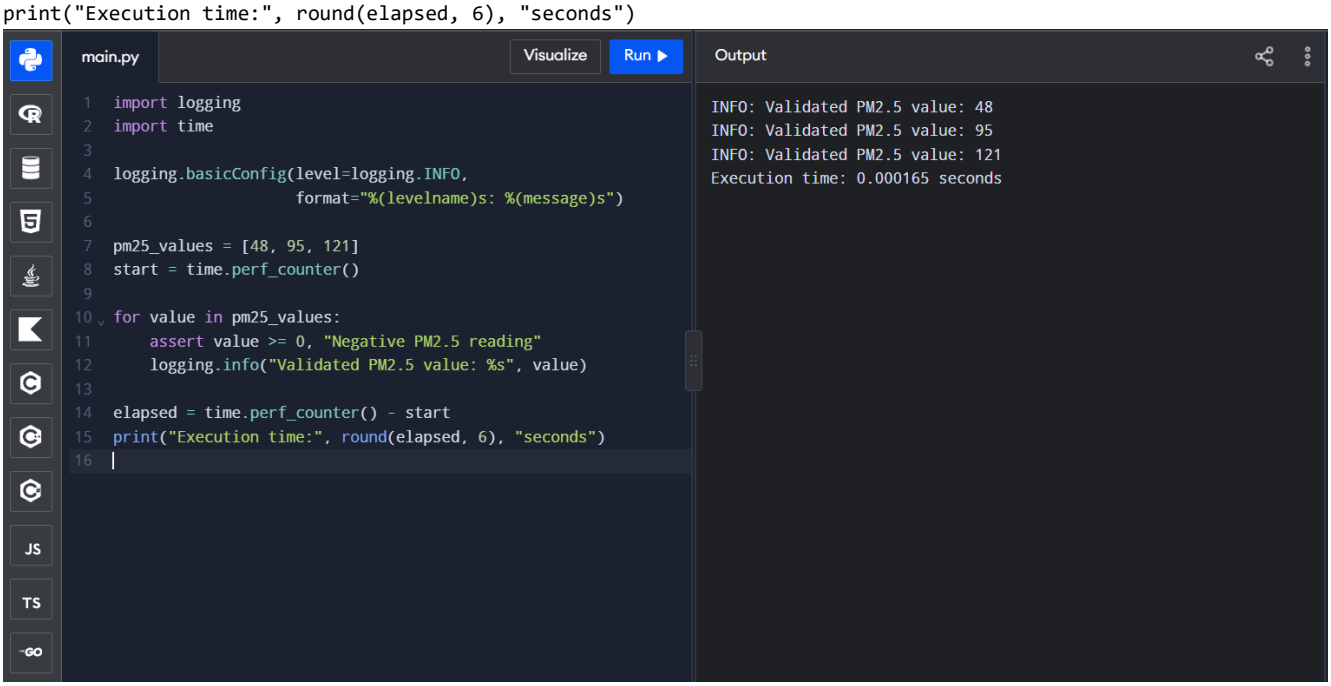
logging.basicConfig(level=logging.INFO,
                    format="%(levelname)s: %(message)s")

pm25_values = [48, 95, 121]
start = time.perf_counter()

for value in pm25_values:
    assert value >= 0, "Negative PM2.5 reading"
```

```
    logging.info("Validated PM2.5 value: %s", value)

elapsed = time.perf_counter() - start
print("Execution time:", round(elapsed, 6), "seconds")
```



TASK 5 – DESIGN AN AI APPLICATION FOR CITIZEN COMPLAINTS (A5)

A5.1 – Sentiment and Summary

Sentiment analysis identifies whether a citizen complaint is positive, negative or neutral.

- Positive words: good, clean, helpful, improved.
- Negative words: bad, dirty, slow, smoke.
- Negation words: not, never, no.
- Example: 'The air is clean' – Positive.
- Example: 'The air is dirty' – Negative.
- Example: 'The service is not good' – Negative.

Extractive summarisation: Divide complaints into sentences, remove common words, count important words, identify sentences containing frequently occurring important words and select the most relevant sentences as the summary.

A5.2 – Basic Chatbot

A rule-based chatbot answers questions using keywords and database queries.

- Receive a citizen's question.
- Identify the station name.
- Identify the required parameter, such as PM2.5.
- Query the SQLite database.
- Display the result.

Example: Question – What is the PM2.5 in Adyar today? Response – The chatbot searches the database for Adyar's latest PM2.5 reading.

Limitations: It understands only predefined keywords, cannot understand every question, may fail with spelling mistakes, cannot understand complex language and requires database connectivity.

Alternative Concept and Complexity Analysis

Feature	Word-list method	Machine-learning method
Training	Not required	Required for Naive Bayes
Memory	Low	Higher
Execution	Usually fast	Depends on model
Accuracy	Limited	Can be higher
Language understanding	Basic	More advanced

A trained model can understand more complex language, but it requires additional memory, training data and computational resources.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

Runnable Python Demonstration – TASK 5 (A5)

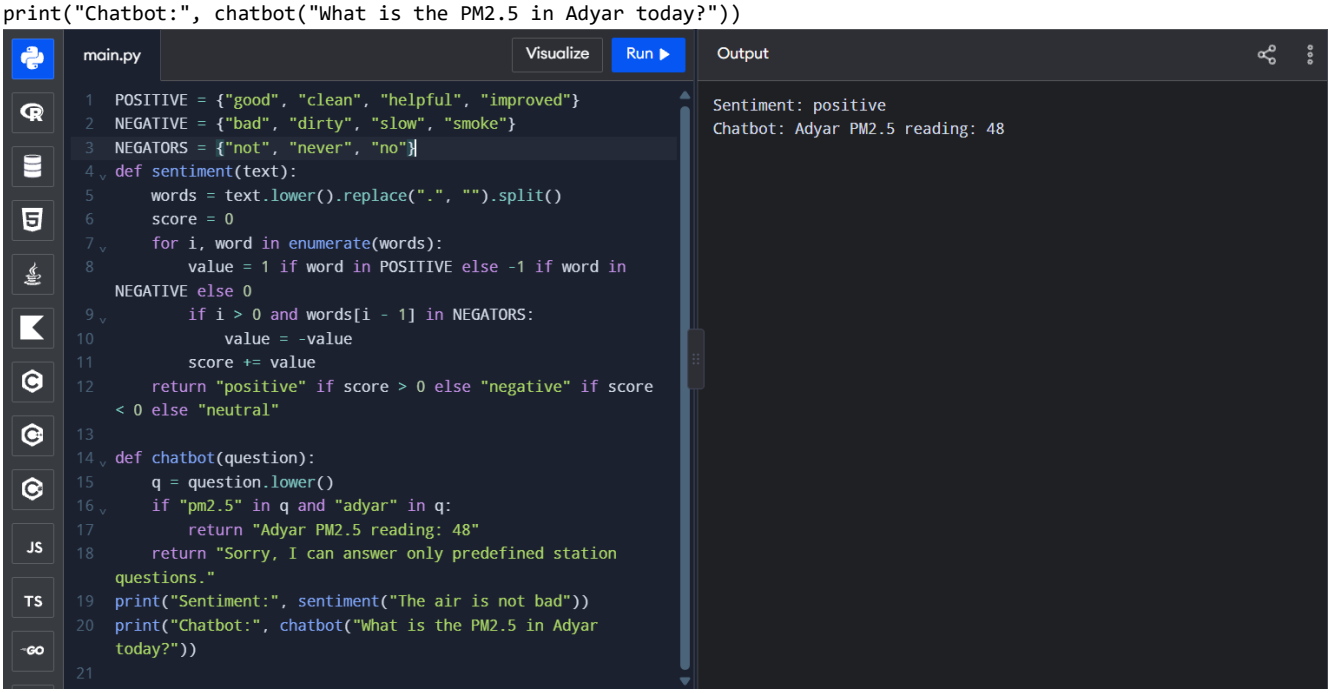
```
# TASK A5 – Python demonstration: lexicon sentiment and keyword chatbot
POSITIVE = {"good", "clean", "helpful", "improved"}
NEGATIVE = {"bad", "dirty", "slow", "smoke"}
NEGATORS = {"not", "never", "no"}

def sentiment(text):
    words = text.lower().replace(".", "").split()
    score = 0
    for i, word in enumerate(words):
        value = 1 if word in POSITIVE else -1 if word in NEGATIVE else 0
        if i > 0 and words[i - 1] in NEGATORS:
            value = -value
        score += value
    return "positive" if score > 0 else "negative" if score < 0 else "neutral"

def chatbot(question):
    q = question.lower()
    if "pm2.5" in q and "adyar" in q:
        return "Adyar PM2.5 reading: 48"
    return "Sorry, I can answer only predefined station questions."

print("Sentiment:", sentiment("The air is not bad"))
```

```
print("Chatbot:", chatbot("What is the PM2.5 in Adyar today?"))
```



TASK 6 – IMPLEMENT AND VALIDATE THE CLEANING PIPELINE (B1)

B1.1 – Validate the Implementation

Aim: To clean air-quality data by removing missing, negative and invalid PM2.5 readings.

Python Program – Complete Code (TODO 1 and TODO 2)

```
import csv
import io

RAW = """station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121
Guindy,2026-10-04,1500
"""

def load_clean(text):
    rows = []
    reader = csv.DictReader(io.StringIO(text))

    for row in reader:
        value = row["pm25"].strip()

        if value == "" or value.upper() == "NA":
            continue

        pm25 = float(value)

        if 0 <= pm25 <= 999:
            rows.append((row["station"], row["date"], pm25))

    return rows
```

```
def min_max(values):
    minimum = min(values)
    maximum = max(values)

    if maximum == minimum:
        return [0.0 for v in values]

    return [
        round((v - minimum) / (maximum - minimum), 3)
        for v in values
    ]

clean = load_clean(RAW)

print("Cleaned Data:")
print(clean)

print("Normalized Values:")
print(min_max([pm for _, _, pm in clean]))
```

Expected Output:

```
Cleaned Data:
[('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)]

Normalized Values:
[0.0, 0.644, 1.0]
```

Explanation: csv.DictReader() reads CSV data. Missing and NA values are skipped. Negative values and values above 999 are removed. Valid values are converted into floating-point numbers. Min-max normalisation converts values into the range 0 to 1.

B1.2 – Compare with Your Design

- The implementation follows the cleaning rules designed in Part A.
- Missing values are removed.
- Negative readings are rejected.
- Extremely large readings are rejected.
- Valid values are converted into floating-point numbers.
- Min-max normalisation is applied successfully.

Conclusion: The implementation matches the original design and produces clean and normalised data.

B1.3 – Store and Query

Python Program – SQLite Database

```
import sqlite3

readings = [
    ("Adyar", "2026-10-01", 48.0),
    ("Guindy", "2026-10-01", 95.0),
    ("Guindy", "2026-10-03", 121.0),
    ("Adyar", "2026-10-04", 60.0)
]

con = sqlite3.connect(":memory:")

con.execute("""
CREATE TABLE aq (
    station TEXT,
    day TEXT,
    pm25 REAL
```

```
)
""")

con.executemany("INSERT INTO aq VALUES (?, ?, ?)", readings)
con.commit()

queries = [("Adyar",), ("Guindy",)]

for q in queries:
    result = con.execute(
        "SELECT AVG(pm25) FROM aq WHERE station = ?", q
    ).fetchone()
    print(q[0], "Average PM2.5:", result[0])

con.close()
```

Expected Output:

```
Adyar Average PM2.5: 54.0
Guindy Average PM2.5: 108.0
```

Explanation: SQLite stores the cleaned readings. executemany() inserts multiple records. AVG() calculates average PM2.5. WHERE filters readings by station.

B1.4 – Visualise

Python Program – Air Quality Charts

```
import matplotlib.pyplot as plt

dates = ["Oct 1", "Oct 2", "Oct 3", "Oct 4"]
pm25 = [48, 55, 65, 60]

stations = ["Adyar", "Guindy", "Velachery"]
values = [54, 108, 75]

temperature = [25, 28, 30, 32, 35]
pollution = [35, 45, 55, 70, 90]

hours = ["6 AM", "9 AM", "12 PM", "3 PM", "6 PM"]
hourly = [45, 65, 50, 80, 95]

plt.figure()
plt.plot(dates, pm25, marker="o")
plt.title("Monthly PM2.5 Trend")
plt.xlabel("Date")
plt.ylabel("PM2.5")
plt.show()

plt.figure()
plt.bar(stations, values)
plt.title("Station Comparison")
plt.xlabel("Station")
plt.ylabel("Average PM2.5")
plt.show()

plt.figure()
plt.scatter(temperature, pollution)
plt.title("Temperature vs PM2.5")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()

plt.figure()
plt.bar(hours, hourly)
```

```
plt.title("Hourly Air Pollution")
plt.xlabel("Hour")
plt.ylabel("PM2.5")
plt.show()
```

Output and Interpretation:

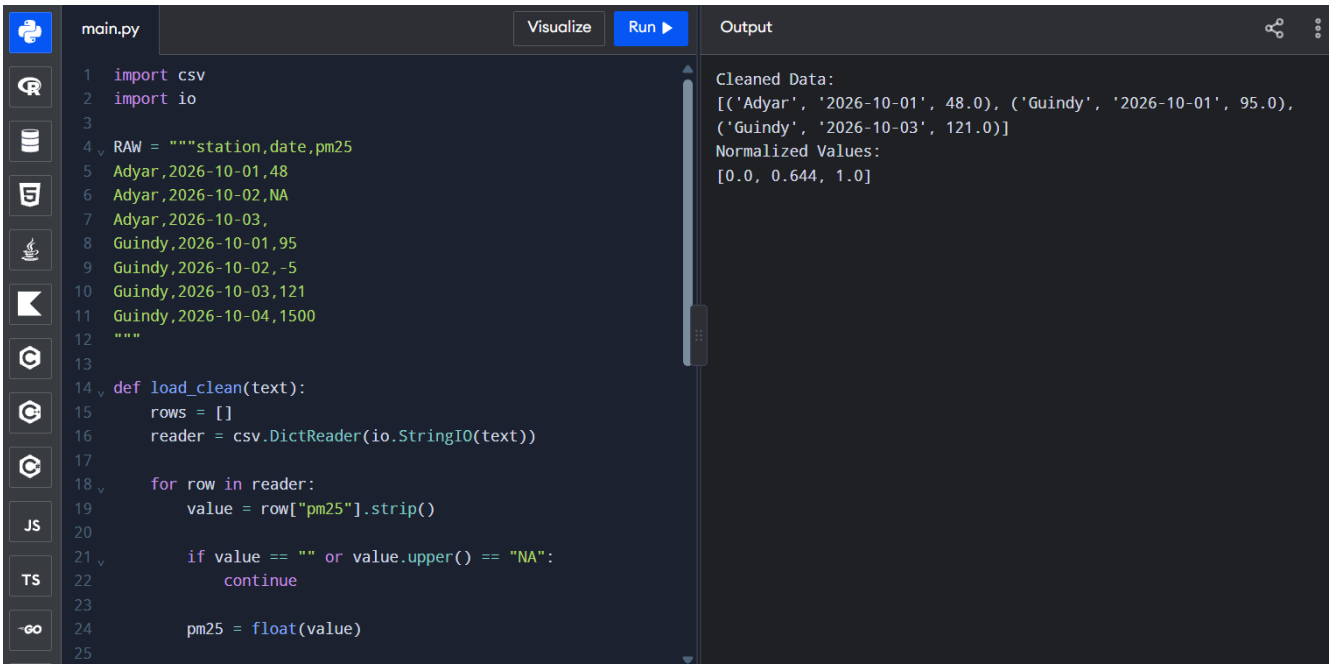
- Monthly PM2.5 Trend: The line chart shows how pollution changes over time.
- Station Comparison: The bar chart shows that Guindy has the highest average reading among the example stations.
- Temperature vs PM2.5: The scatter plot displays the relationship between temperature and pollution.
- Hourly Air Pollution: The bar chart identifies the hours with higher pollution.

Alternative Concept and Complexity Analysis

Number of rows	Expected processing behaviour
1,000	Very fast
100,000	Moderate processing
1,000,000	Higher memory requirement

- CSV loops: O(n).
- Pandas cleaning: O(n) for typical vectorised operations.
- Space: O(n) for loaded data.

Pandas is convenient for large-scale data manipulation, but it may consume more memory because the dataset is loaded into a DataFrame.



TASK 7 – ENGINEER ADVERSARIAL DATA (B2)

B2.1 – Test Different Adversarial Inputs

- Missing column: The input file does not contain the PM2.5 column. The program should identify the missing column and display an error.
- Wrong delimiter: A CSV file uses semicolons instead of commas. The program may read all values as one column. The delimiter must be detected or configured correctly.

- Different text encoding: A file uses an encoding such as Latin-1. Reading with an incompatible encoding may cause an error. Specify the correct encoding.
- Empty file: The input file contains no records. The program should display an empty-data warning.

B2.2 – SQL Injection Testing

Using string concatenation:

```
name = "Adyar' OR '1'='1"
query = "SELECT * FROM aq WHERE station = '" + name + "'"
```

This can change the SQL condition and potentially return unintended records.

Using parameterised queries:

```
query = "SELECT * FROM aq WHERE station = ?"
```

The input is treated as a value, not executable SQL code. Parameterised queries provide better security and prevent SQL injection.

B2.3 – Sentiment Testing

Complaint	Expected result
Air is clean	Positive
Roads are dirty	Negative
Not good service	Negative
Not bad service	Positive
Good service but dirty roads	Mixed

- Sarcasm may be misunderstood.
- Double negatives may produce incorrect results.
- Mixed opinions may be classified incorrectly.
- The word-list method cannot fully understand context.

B2.4 – Explanation of Adversarial Inputs

- Missing columns test schema validation.
- Wrong delimiters test CSV handling.
- Different encodings test file compatibility.
- Empty files test exception handling.
- SQL injection tests database security.
- Sarcasm and mixed opinions test sentiment-analysis limitations.

Complete the Code – TODO 1 and TODO 2

Python Program:

```
import sqlite3

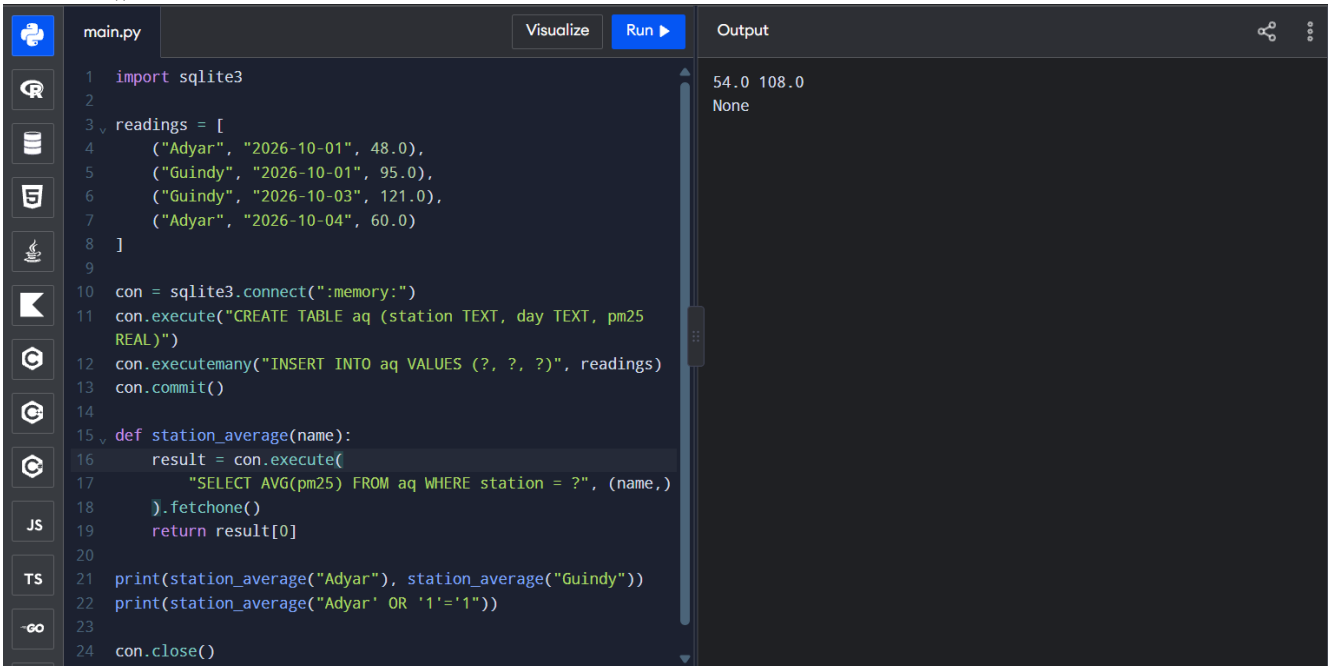
readings = [
    ("Adyar", "2026-10-01", 48.0),
    ("Guindy", "2026-10-01", 95.0),
    ("Guindy", "2026-10-03", 121.0),
    ("Adyar", "2026-10-04", 60.0)
]

con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE aq (station TEXT, day TEXT, pm25 REAL)")
con.executemany("INSERT INTO aq VALUES (?, ?, ?)", readings)
con.commit()
```

```
def station_average(name):
    result = con.execute(
        "SELECT AVG(pm25) FROM aq WHERE station = ?", (name,)
    ).fetchone()
    return result[0]

print(station_average("Adyar"), station_average("Guindy"))
print(station_average("Adyar' OR '1'='1"))

con.close()
```



Expected Output:

```
54.0 108.0
None
```

The input Adyar' OR '1'='1 is treated as a normal string because the query uses a parameterised placeholder. No station has that exact name, so the query returns None instead of all database records.

Alternative Concept and Complexity Analysis

String concatenation can allow special characters to change SQL queries and create SQL injection risks. Parameterised queries are safer. executemany() generally reduces Python-to-SQLite call overhead compared with repeated execute() calls in a loop.

TASK 8 – LOOPS VS LIBRARIES VS SQL (B3)

B3.1 – Correctness on Small Data

The daily average PM2.5 is calculated using three approaches: Python loops with the CSV module, pandas groupby and SQL GROUP BY.

Python Program – Three Methods Comparison

```
import csv
import io
import sqlite3
from collections import defaultdict
import pandas as pd

data = """station,day,pm25
```

```
Adyar,2026-10-01,48
Adyar,2026-10-01,60
Guindy,2026-10-01,95
Guindy,2026-10-01,105
Adyar,2026-10-02,70
"""

# Method 1: Python loops
reader = csv.DictReader(io.StringIO(data))
groups = defaultdict(list)

for row in reader:
    key = (row["station"], row["day"])
    groups[key].append(float(row["pm25"]))

print("Python Loops:")
for key, values in sorted(groups.items()):
    print(key, sum(values) / len(values))

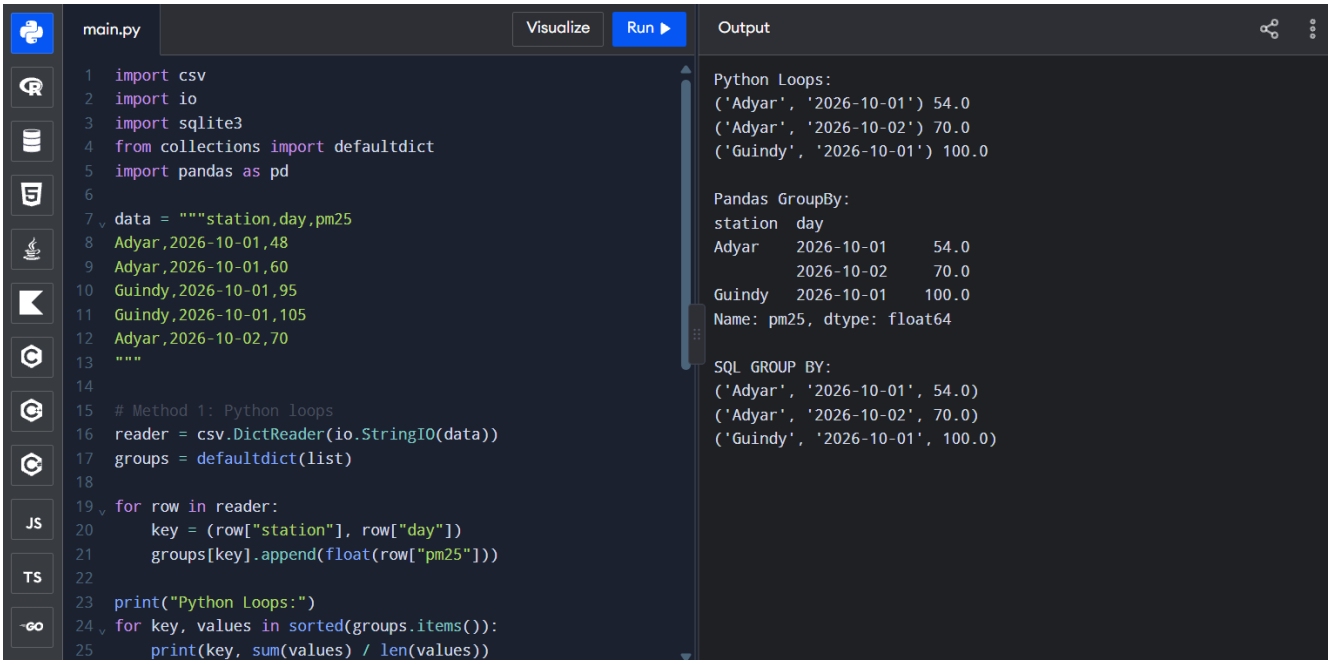
# Method 2: Pandas
df = pd.read_csv(io.StringIO(data))
print("\nPandas GroupBy:")
print(df.groupby(["station", "day"])["pm25"].mean())

# Method 3: SQL
con = sqlite3.connect(":memory:")
df.to_sql("aq", con, index=False)

print("\nSQL GROUP BY:")
result = con.execute("""
SELECT station, day, AVG(pm25)
FROM aq
GROUP BY station, day
ORDER BY station, day
""")

for row in result:
    print(row)

con.close()
```



Expected Output:

```
Python Loops:
('Adyar', '2026-10-01') 54.0
('Adyar', '2026-10-02') 70.0
('Guindy', '2026-10-01') 100.0

Pandas GroupBy:
station  day
Adyar    2026-10-01     54.0
         2026-10-02     70.0
Guindy   2026-10-01    100.0

SQL GROUP BY:
('Adyar', '2026-10-01', 54.0)
('Adyar', '2026-10-02', 70.0)
('Guindy', '2026-10-01', 100.0)
```

B3.2 – Increase the Data Size

The program below generates data for 1, 12 and 120 stations and measures processing time.

Python Program – Performance Measurement

```
import time
import random
import pandas as pd
import sqlite3
from collections import defaultdict

random.seed(10)

for station_count in [1, 12, 120]:
    data = []

    for s in range(station_count):
        for day in range(365):
            for hour in range(24):
                data.append((f"Station{s}", day, random.randint(10, 150)))

    print("\nStations:", station_count)
    print("Total readings:", len(data))

    start = time.perf_counter()
    groups = defaultdict(list)
    for station, day, value in data:
        groups[(station, day)].append(value)
    loop_result = {
        key: sum(values) / len(values)
        for key, values in groups.items()
    }
    loop_time = time.perf_counter() - start

    start = time.perf_counter()
    df = pd.DataFrame(data, columns=["station", "day", "pm25"])
    pandas_result = df.groupby(["station", "day"])["pm25"].mean()
    pandas_time = time.perf_counter() - start

    start = time.perf_counter()
    con = sqlite3.connect(":memory:")
    con.execute("CREATE TABLE aq (station TEXT, day INTEGER, pm25 REAL)")
    con.executemany("INSERT INTO aq VALUES (?, ?, ?)", data)
    sql_result = con.execute("""
        SELECT station, day, AVG(pm25)
```

```
        FROM aq
        GROUP BY station, day
    """).fetchall()
    sql_time = time.perf_counter() - start

    print("Python time:", round(loop_time, 4), "seconds")
    print("Pandas time:", round(pandas_time, 4), "seconds")
    print("SQL time:", round(sql_time, 4), "seconds")
    con.close()
```

Expected Output: The program displays the number of stations, total readings, Python execution time, pandas execution time and SQL execution time. Actual timings vary depending on the computer and installed library versions. Use the values printed by your execution rather than invented timing values.

B3.3 – Profile the Plain-Python Version

Python Program – cProfile

```
import cProfile
import random
from collections import defaultdict

def calculate_average():
    groups = defaultdict(list)

    for station in range(12):
        for day in range(365):
            for hour in range(24):
                value = random.randint(10, 150)
                groups[(station, day)].append(value)

    result = {}
    for key, values in groups.items():
        result[key] = sum(values) / len(values)

    return result

cProfile.run("calculate_average()")
```

Output: The program displays a profiling report containing function name, number of calls, total execution time and cumulative execution time.

Interpretation: The loops generating and grouping readings consume a significant part of processing time. The profiler helps identify functions that need optimisation.

B3.4 – Engineering Trade-Off

- Quick one-time analysis: Pandas is suitable because it provides simple functions for filtering, grouping and analysing data.
- Daily automated reporting: SQLite is suitable because it provides persistent storage, SQL queries and indexing.

Method	Advantage	Limitation
Python loops	Simple and flexible	More manual coding
Pandas	Easy data analysis	Higher memory usage
SQL	Efficient structured queries	Requires database knowledge

Alternative Concept and Complexity Analysis

If running sums and counts are maintained in a dictionary, each reading is processed once. The sum and count are updated, and the average is calculated at the end.

- Time complexity: O(n).
- Space complexity when storing all readings: O(n).
- Space complexity using running sum and count: O(g), where g is the number of station-day groups.

The running sum and count method saves memory because it does not need to store every reading.

TASK 9 – FINAL ENGINEERING DECISION (B4)

B4.1 – Implement a Practical Strategy

Aim: To implement sentiment analysis and identify positive and negative citizen complaints.

Python Program – Complete Code (TODO 1)

```
POSITIVE = {"good", "clean", "fast", "helpful", "improved"}
NEGATIVE = {"bad", "dirty", "slow", "rude", "smoke", "broken"}
NEGATORS = {"not", "never", "no"}

def score(text):
    words = text.lower().replace(".", " ").replace(",", " ").split()
    total = 0

    for i, word in enumerate(words):
        value = 0
        if word in POSITIVE:
            value = 1
        elif word in NEGATIVE:
            value = -1

        if i > 0 and words[i - 1] in NEGATORS:
            value = -value

        total += value

    return total

def label(text):
    s = score(text)
    return (
        "positive" if s > 0
        else "negative" if s < 0
        else "neutral"
    )

complaints = [
    "Roads are clean and the bus was fast.",
    "Smoke from the factory, air is bad.",
    "The staff were not helpful.",
    "Not bad, service improved."
]

for c in complaints:
    print(f"{label(c):8} | {c}")
```

Expected Output:

```
positive | Roads are clean and the bus was fast.
negative | Smoke from the factory, air is bad.
negative | The staff were not helpful.
positive | Not bad, service improved.
```

Explanation: Positive words increase the score. Negative words decrease the score. Negation words reverse the following word's sentiment. The final score determines the sentiment label.

Find a complaint that the scorer labels wrongly: Example – 'The air is not bad.' The scorer labels this complaint as positive because 'not' reverses the negative word 'bad'. However, more complex sentences containing sarcasm or multiple opinions may be classified incorrectly.

Agreement Measurement: Manually label 50 complaints, run the sentiment program, compare predicted labels with manual labels and count matching labels. Agreement = (Matching labels / 50) × 100. For example, if 42 labels match, agreement = 84%. This is an example calculation, not an actual measured result.

B4.2 – Check the Database

Python Program – Create Index and Measure Query Time

```
import sqlite3
import time

con = sqlite3.connect(":memory:")

con.execute("""
CREATE TABLE aq (
    station TEXT,
    day TEXT,
    pm25 REAL
)
""")

data = [
    (f"Station{i % 100}", f"2026-10-{i % 28 + 1:02d}", i % 150)
    for i in range(100000)
]

con.executemany("INSERT INTO aq VALUES (?, ?, ?)", data)
con.commit()

query = """
SELECT AVG(pm25)
FROM aq
WHERE station = ? AND day = ?
"""

start = time.perf_counter()
for i in range(100):
    con.execute(query, ("Station10", "2026-10-11")).fetchone()
before = time.perf_counter() - start

con.execute("CREATE INDEX idx_station_day ON aq(station, day)")
con.commit()

start = time.perf_counter()
for i in range(100):
    con.execute(query, ("Station10", "2026-10-11")).fetchone()
after = time.perf_counter() - start

print("Before Index:", round(before, 5), "seconds")
print("After Index:", round(after, 5), "seconds")

con.close()
```

Expected Output: The program displays two measured execution times: Before Index and After Index. Actual values depend on the computer. The index is expected to improve queries filtering by station and date, although the measured improvement depends on the dataset and query plan.

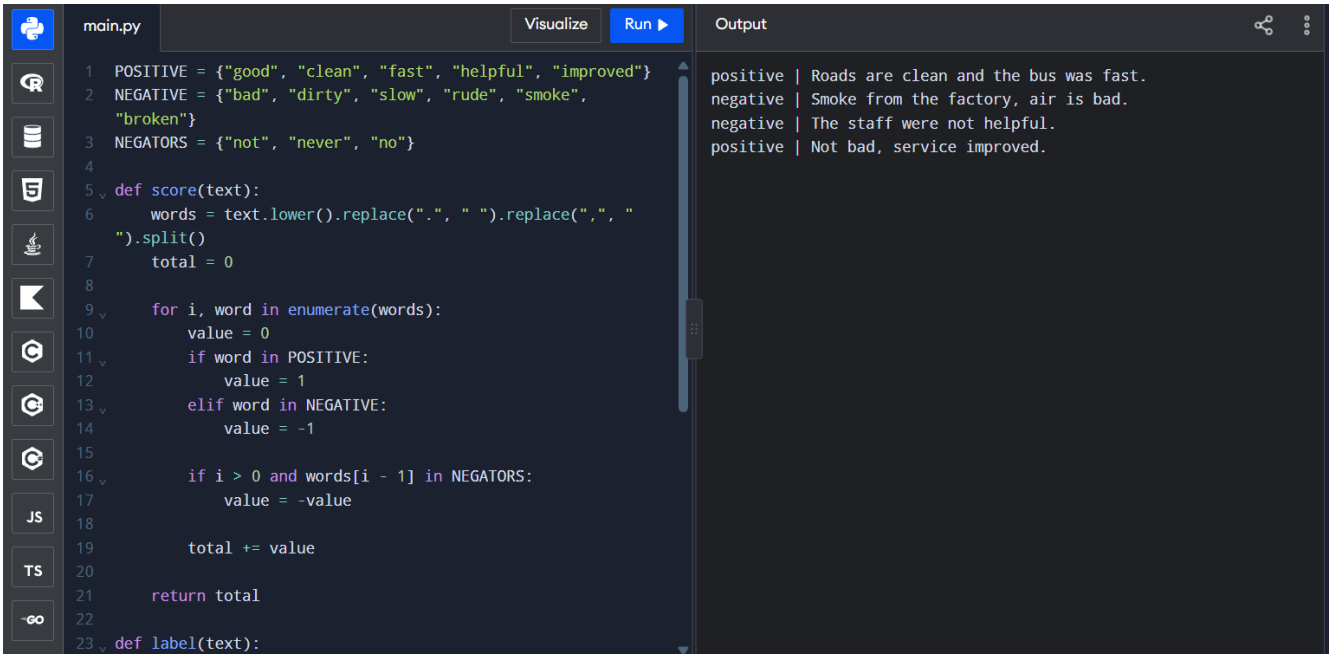
Explanation: An index improves data searching. The composite index contains station and date. It reduces the number of records that need to be examined and improves repeated queries.

B4.3 – Production Decision

The city air-quality monitoring network will add 100 stations and receive 50,000 complaints every month.

- Data Storage: Use SQLite for a small prototype. Use PostgreSQL for a larger multi-user production system. Store station information and readings in separate tables. Create indexes on station and date.
- Data Cleaning: Remove invalid and duplicate records. Handle missing readings. Convert Fahrenheit into Celsius. Standardise dates and numerical values.
- Data Analysis: Use SQL GROUP BY for daily and monthly summaries. Use pandas for detailed analysis. Use Matplotlib to generate charts. Use profiling tools to monitor performance.
- AI Application: Use lexicon-based sentiment analysis for basic complaints. Use machine learning when more complex language understanding is required. Train the model using labelled citizen complaints. Evaluate its accuracy before production use.

Final Decision: A combination of PostgreSQL, Python, pandas and machine-learning methods is suitable for the expanded system. This approach provides reliable storage, efficient analysis, better scalability and improved complaint understanding.



TASK 10 – ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS (B4)

If the summariser selects the first k sentences instead of selecting sentences containing frequent important words:

First-k Sentence Method

- Split the complaint into sentences.
- Select the first k sentences.
- Display them as the summary.

Important-word Summarisation Method

- Split the complaint into sentences.
- Identify important words.
- Calculate word frequencies.
- Select sentences containing frequent important words.

Feature	First-k method	Important-word method
Selection	First k sentences	Relevant sentences

Processing	Simple	More complex
Quality	May miss important information	Usually more informative
Time	O(k) after sentence splitting	O(words) or more, depending on ranking
Space	O(k) output space	O(words) for frequency counting

Conclusion: Important-word summarisation provides more meaningful summaries because it selects sentences based on their content rather than their position.

FINAL CONCLUSION

The assignment demonstrates how Python can be used to acquire, clean, store, analyse and visualise air-quality data.

The implementation uses CSV, SQLite, pandas and Matplotlib. Debugging and profiling techniques help improve program performance. Sentiment analysis and text summarisation provide basic AI applications for understanding citizen complaints.

Complete workflow: Acquire → Clean → Store → Query → Visualise → Profile → Challenge → Interpret.

FINAL SUBMISSION CHECKLIST

- Part A – Design answers (Tasks 1–5).
- Part B – Implementation answers (Tasks 6–10).
- All completed Python programs.
- Input CSV files.
- Program output screenshots.
- Charts and profiling results.
- SQLite database or database creation script.
- README.md.
- requirements.txt.
- GitHub repository link.

Required GitHub repository naming format: CSA08-<RegisterNumber>-Assignment5

Libraries used: pandas and matplotlib. The other libraries used (csv, io, sqlite3, time, random, collections and cProfile) are included in Python's standard library.

Important note: For programs using pandas and Matplotlib, install those libraries in your online compiler or use Google Colab. The other programs can run with standard Python.

For final submission, use your own actual execution screenshots and measured timings rather than treating illustrative output or example measurements as experimental evidence.